

Phase 0 — Infrastructure Setup

Overview

This document describes the foundational infrastructure for the AIA Platform. Phase 0 establishes the services needed before any application code is written.

Services

PostgreSQL 16 + pgvector

Purpose: Primary relational database for all platform data (tenants, users, conversations, usage tracking). The pgvector extension enables storing and querying vector embeddings directly in PostgreSQL for simpler architectures.

Why PostgreSQL: Battle-tested, excellent JSON support, pgvector eliminates the need for a separate vector DB in simple cases, strong ecosystem.

Configuration:

- Image: pgvector/pgvector:pg16 (official pgvector build on PG16)
- Extensions: uuid-oss, vector, pg_trgm, unaccent
- Volume: postgres-data for persistence
- Init scripts run once on first start

Redis 7

Purpose: Caching layer, session storage, rate limiting counters, and background job queues (BullMQ).

Why Redis: Sub-millisecond latency, built-in data structures (sorted sets for rate limiting, streams for queues), AOF persistence for durability.

Configuration:

- AOF persistence with everysec fsync
- 512MB memory limit with LRU eviction
- Dangerous commands (FLUSHDB, FLUSHALL) disabled

Qdrant

Purpose: Dedicated vector database for knowledge base semantic search. Handles large-scale embedding storage and approximate nearest neighbor (ANN) queries.

Why Qdrant: Purpose-built for vector search, better performance than pgvector at scale (>100K vectors), filtering capabilities, gRPC support for low latency.

Configuration:

- HTTP API on port 6333, gRPC on 6334
- WAL enabled for durability
- Telemetry disabled

LiteLLM

Purpose: AI model gateway that provides a unified OpenAI-compatible API across multiple LLM providers. Handles routing, fallbacks, cost tracking, and rate limiting.

Why LiteLLM: Single interface for all models, automatic retries/fallbacks, built-in cost tracking, per-key budget limits, OpenAI-compatible (easy client implementation).

Model Tiers:

Tier	Model	Use Case	Approx. Cost
fast-cheap	DeepSeek Chat	Simple Q&A, classification, extraction	~\$0.14/1M tokens
balanced	Claude 3.5 Haiku	Complex reasoning, analysis, writing	~\$1/1M input
powerful	Claude Sonnet 4	Critical tasks, complex analysis	~\$3/1M input
embedding	text-embedding-3-small	Document embeddings for RAG	~\$0.02/1M tokens

Traefik v3

Purpose: Reverse proxy with automatic SSL certificate provisioning via Let's Encrypt.

Why Traefik: Docker-native (auto-discovers services via labels), automatic HTTPS, built-in metrics, middleware support (rate limiting, auth, headers).

Configuration:

- HTTP to HTTPS redirect
- Let's Encrypt via HTTP challenge
- Security headers middleware
- Access logging in JSON format

Local Development Setup

Prerequisites

- Docker Desktop (or Docker Engine + Compose v2)
- Node.js 20+ and pnpm
- API keys for at least one AI provider (minimum: DEEPSEEK_API_KEY)

Quick Start — One Command

```
# 1. Clone and enter
git clone <repo-url> aia-platform
cd aia-platform

# 2. Create .env
cp .env.example .env
# Edit: DEEPSEEK_API_KEY=sk-... and LITELLM_MASTER_KEY=sk-aia-local-dev-123

# 3. Start EVERYTHING
make dev
```

This single command:

- Runs pre-flight checks (Docker, Node, .env, API keys)
- Starts Docker infrastructure (PostgreSQL, Redis, Qdrant, LiteLLM, Neo4j)
- Waits for all services to be healthy
- Installs pnpm dependencies (if needed)
- Builds shared packages (@aia/shared, @aia/ai-client, @aia/auth, @aia/graph)
- Starts Gateway API (port 3000), Dashboard (port 5173), Client (port 5174)

Other dev commands

```

make dev-status # Full status with health checks
make dev-stop   # Stop everything (Docker + apps)
make dev-skip   # Start apps only (Docker already running)

```

Manual Step-by-Step (if you prefer control)

```

# 1. Start Docker infrastructure only
make up

# 2. Verify services
make status
make llm-health
curl http://localhost:6333/healthz

# 3. Install and build
pnpm install
pnpm --filter @aia/shared build
pnpm --filter @aia/ai-client build

# 4. Start gateway
cd apps/gateway && pnpm dev

# 5. Start dashboard (separate terminal)
cd apps/dashboard && pnpm dev

# 6. Start client (separate terminal)
cd apps/client && pnpm dev

```

Verify everything works

```

# Database
make psql
# \dn          -- should show 'shared' schema
# \dt shared.* -- should show all tables
# SELECT * FROM shared.plans; -- should show 3 plans
# \q

# Redis
make redis
# PING        -- should return PONG

# Qdrant
curl http://localhost:6333/healthz

# LiteLLM
make llm-health

# AI model test (requires valid API key)
make llm-test

# Gateway API
curl http://localhost:3000/health

```

Troubleshooting

PostgreSQL won't start

```
# Check logs
docker compose logs postgres

# Common: port 5432 already in use
# Fix: stop local PostgreSQL or change port in docker-compose.dev.yml
ls -l /etc/passwd
```

LiteLLM unhealthy

```
# Check logs (usually API key issues)
docker compose logs litellm

# Common issues:
# - Invalid API key format
# - Database not ready (wait for postgres healthcheck)
# - Config syntax error in config.yaml
```

Redis connection refused

```
# Check if redis.conf syntax is valid
docker compose logs redis

# Common: custom config has syntax error
# Fix: validate with `redis-server --test-memory 1` inside container
```

Qdrant not reachable

```
# Check logs
docker compose logs qdrant

# Common: config.yaml indentation error
# Qdrant is strict about YAML format
```

Out of disk space

```
# Check Docker disk usage
docker system df

# Clean unused images and volumes
docker system prune -a --volumes
```

Health Verification Checklist

After make up, verify each service:

Service	Check Command	Expected Result
PostgreSQL	make psql then SELECT 1;	Returns 1
Redis	make redis then PING	Returns PONG
Qdrant	curl localhost:6333/healthz	JSON with version
LiteLLM	curl localhost:4000/health	{"status":"healthy"}
All	make status	All containers Up (healthy)

Production Deployment (Hetzner VPS)

Recommended VPS Specs

Resource | Minimum | Recommended

CPU | 2 vCPU (shared) | 4 vCPU (dedicated)

RAM | 4 GB | 8 GB

Storage | 40 GB NVMe | 80 GB NVMe

Traffic | 20 TB included | 20 TB included

Recommended plan: Hetzner CX32 (4 vCPU, 8 GB RAM, 80 GB) at ~EUR 13/month.

Deployment Steps

```
# 1. Provision VPS via Hetzner Cloud Console
# - Ubuntu 24.04
# - Add your SSH key
# - Location: Nuremberg or Falkenstein (EU)

# 2. Run setup script on VPS
scp scripts/setup-vps.sh root@YOUR_IP:/root/
ssh root@YOUR_IP 'bash /root/setup-vps.sh'

# 3. Clone repo as 'aia' user
ssh aia@YOUR_IP
cd /opt/aia-platform
git clone <repo-url> .

# 4. Configure environment
cp .env.example .env
nano .env
# IMPORTANT: Change ALL passwords and secrets
# Set DOMAIN to your actual domain
# Set ACME_EMAIL for Let's Encrypt

# 5. Point DNS to VPS IP
# A record: aia.yourdomain.com -> VPS_IP
# A record: llm.aia.yourdomain.com -> VPS_IP (for LiteLLM)
# A record: traefik.aia.yourdomain.com -> VPS_IP (for dashboard)

# 6. Start production stack (with Traefik)
docker compose up -d

# 7. Verify SSL is working
curl https://aia.yourdomain.com
```

Ongoing Deployment

```
# From your local machine:
./scripts/deploy.sh

# Or manually on VPS:
ssh aia@YOUR_IP
cd /opt/aia-platform
git pull
docker compose up -d --remove-orphans
```

Backups

Automated daily backups via cron:

```
# On VPS, as 'aia' user:
crontab -e
# Add:
0 3 * * * cd /opt/aia-platform && ./infrastructure/backups/backup.sh --upload >> /var/log/aia-backup.log 2>&1
```

Cost Breakdown

Infrastructure (Monthly)

Item	Cost (EUR)
Hetzner CX32 VPS	~13
Domain (amortized)	~1
S3 backup storage (10 GB)	~0.25
Total infrastructure	~14.25

AI API Costs (Variable, per tenant)

Tier	Cost per 1M tokens	Typical monthly (per tenant)
fast-cheap (DeepSeek)	~\$0.14 input, \$0.28 output	\$5-20
balanced (Haiku)	~\$1 input, \$5 output	\$20-80
powerful (Sonnet)	~\$3 input, \$15 output	\$50-200
embedding	~\$0.02	\$1-5

Margin Calculation

With the starter plan at EUR 300/month:

- Infrastructure cost per tenant: ~EUR 2 (14.25 / ~7 tenants on one VPS)
- AI cost per tenant: ~EUR 20-50 (fast-cheap tier, 200 conversations)
- Gross margin: ~80-90%

Architecture Decisions

ADR-001: LiteLLM as AI Gateway

Context: Need to support multiple AI providers with unified interface.

Decision: Use LiteLLM as a centralized proxy rather than direct SDK calls.

Consequences:

- (+) Single OpenAI-compatible interface for all models
- (+) Built-in cost tracking and budget enforcement
- (+) Easy model switching without code changes
- (+) Retry/fallback logic centralized
- (-) Additional network hop (~5ms latency)
- (-) Dependency on LiteLLM project maintenance

ADR-002: PostgreSQL + Qdrant (dual vector strategy)

Context: Need vector search for knowledge base RAG.

Decision: Use pgvector for small collections (<10K vectors per tenant), Qdrant for large-scale search.

Consequences:

- (+) Simple tenants don't need Qdrant at all

- (+) Large knowledge bases get dedicated vector performance
- (-) Two systems to maintain
- (-) Need routing logic to decide which to use

ADR-003: Multi-tenant shared database

Context: How to isolate tenant data.

Decision: Shared database with `tenant_id` column on all tables (row-level isolation).

Consequences:

- (+) Simple operations, single connection pool
- (+) Easy cross-tenant reporting for platform admin
- (-) Must enforce tenant filtering on EVERY query (risk of data leak if missed)
- (-) Noisy neighbor risk (mitigated by plan limits and rate limiting)

Next Phases

- Phase 1: Gateway API (auth, tenant management, chat endpoint)
- Phase 2: Knowledge base ingestion pipeline
- Phase 3: Dashboard (admin UI, analytics)
- Phase 4: Client widget (embeddable chat)
- Phase 5: Billing integration, production hardening